

MEDIATEK

MT8676 Hypervisor Display User Manual

版本: 1.3
出版日期: 2025-07-11

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-10-28	王煜	正式版
1.1	2024-11-13	王煜	- 在章节 1.2.3.2 Hypervisor (Yocto + Android + Tbox)中 添加 Tbox - 添加表 1 3. Connector 设备节点说明
1.2	2025-06-13-	王煜	添加章节 1.3.2 dispsys_config 节点配置
1.3	2025-07-11	王煜	- 添加章节 1.2.1 模块简介 - 在章节 1.2.3.1 Android 中添加 HWC 图层合成策略 介绍 - 添加章节 1.4.1 Dump 方法

目录

版本记录 2

目录 3

图片目录 3

表格目录 4

1 显示 & 多屏 5

 1.1 概述 5

 1.1.1 基本概述 5

 1.1.2 缩略词 5

 1.1.3 性能 5

 1.2 架构/流程概要 6

 1.2.1 模块简介 6

 1.2.2 HW 架构 7

 1.2.3 SW 架构 8

 1.3 配置/客制指南 11

 1.3.1 设备节点配置 11

 1.3.2 dispsys_config 节点配置 13

 1.4 常见问题/故障排除 21

 1.4.1 Dump 方法 21

 1.4.2 HWC CMD 21

 1.4.3 显示驱动 CMD 22

 1.4.4 Pattern 22

 1.4.5 黑屏 23

 1.4.6 花屏/闪屏 23

附件一 附加条款 24

图片目录

图 1-1. Display HW 架构图 7

图 1-2. Display Android 架构图 8

图 1-3. Android Display 流程 9

图 1-4. Display 多系统架构图10

图 1-5. Display 硬件框图13

图 1-6. Dump 方法总览21

表格目录

表 1-1. 缩略词5

表 1-2. 性能5

表 1-3. Connector 设备节点说明11

1 显示 & 多屏

1.1 概述

1.1.1 基本概述

本章节主要介绍 MT8676 display hypervisor 架构以及多屏配置指南。

1.1.2 缩略词

表 1-1. 缩略词

缩略词	全称	释义
CRTC	Cathode Ray Tube Controller	阴极射线管控制器（即显示控制器）
DP	Display Port	数字式视频接口标准
DPI	Digital Parallel Interface	数字并行接口
DSC	Display Stream Compression	显示流压缩
DSI	Display Serial Interface	显示串行接口
DTS	Device Tree	设备树
HWC	Hardware Composer	硬件合成器
LCM	Liquid Crystal Display Module	液晶显示模组
SerDes	Serializer/Deserializer	加串器/解串器

1.1.3 性能

表 1-2. 性能

名称	性能
DISP pipeline	10bit pipe x 3 (MAX 688MHz at 0.75V)
DSI	DSIO + DSI1 C/D PHY Combo 4-lane x 2 DPHY: 2.5Gbps/lane
DP	DP1.4, 4-lane 8.1Gbps/lane (4-lane mode conflict with USB3)
Panel number	1 ~ 6

名称	性能
OVL layers	OVL0~OVL7 : Total 16 layers
OVL De-comp.	AFBC (RGBA8888/RGB888/RGB565)
Compression	VESA DSC 1.2 (2 slice x 2)

1.2 架构/流程概要

1.2.1 模块简介

HWC (Hardware Composer): Android 图形系统中的核心组件之一, 主要负责屏幕内容的合成与显示, 参考 <https://source.android.com/docs/core/graphics/hwc>。

Wayland: Wayland 是一种现代化的显示服务器协议和架构, 为 Linux 和类 Unix 操作系统提供更高效、更安全、更简洁的图形显示解决方案。参考 <https://wayland.freedesktop.org/>。

PQService (Picture Quality Enhancement Service): 用于支持 MediaTek 独创的 MiraVision 图像显示增强技术, 可以根据观看内容及周围环境 (如强光下) 多方面智能调整显示及视频串流, 包括画面的色彩、亮度、对比度、锐利度、动态范围等, 提升整体显示画质。

MTK DRM Driver: 是 MediaTek 平台上用于显示子系统基于 DRM 的内核驱动, 核心作用是管理和控制显示资源, 实现高效的图层合成和显示输出。参考 <https://dri.freedesktop.org/docs/drm/>。

OVL (Overlay): 显示子系统中负责叠图的硬件模块。每个 OVL 可以对两个图层进行合成, 分别是 Layer0 和 Layer1, Layer1 的 z-order 比 Layer0 高。多个 OVL 串接起来可支持更多图层的合成, 比如 2 个 OVL 串接起来可以支持 4 个图层合成。

CMDQ (Command Queue): 用于驱动 Global Command Engine (GCE), GCE 是 Multimedia 子系统专用的访问硬件寄存器的硬件模块, 相较于通过 CPU 的访问方式, CMDQ 具有更好的性能。

MML/MDP: 显示子系统中负责视频图层处理的硬件模块, 包括缩放、旋转、色彩格式转换、显示效果增强等。

1.2.2 HW 架构

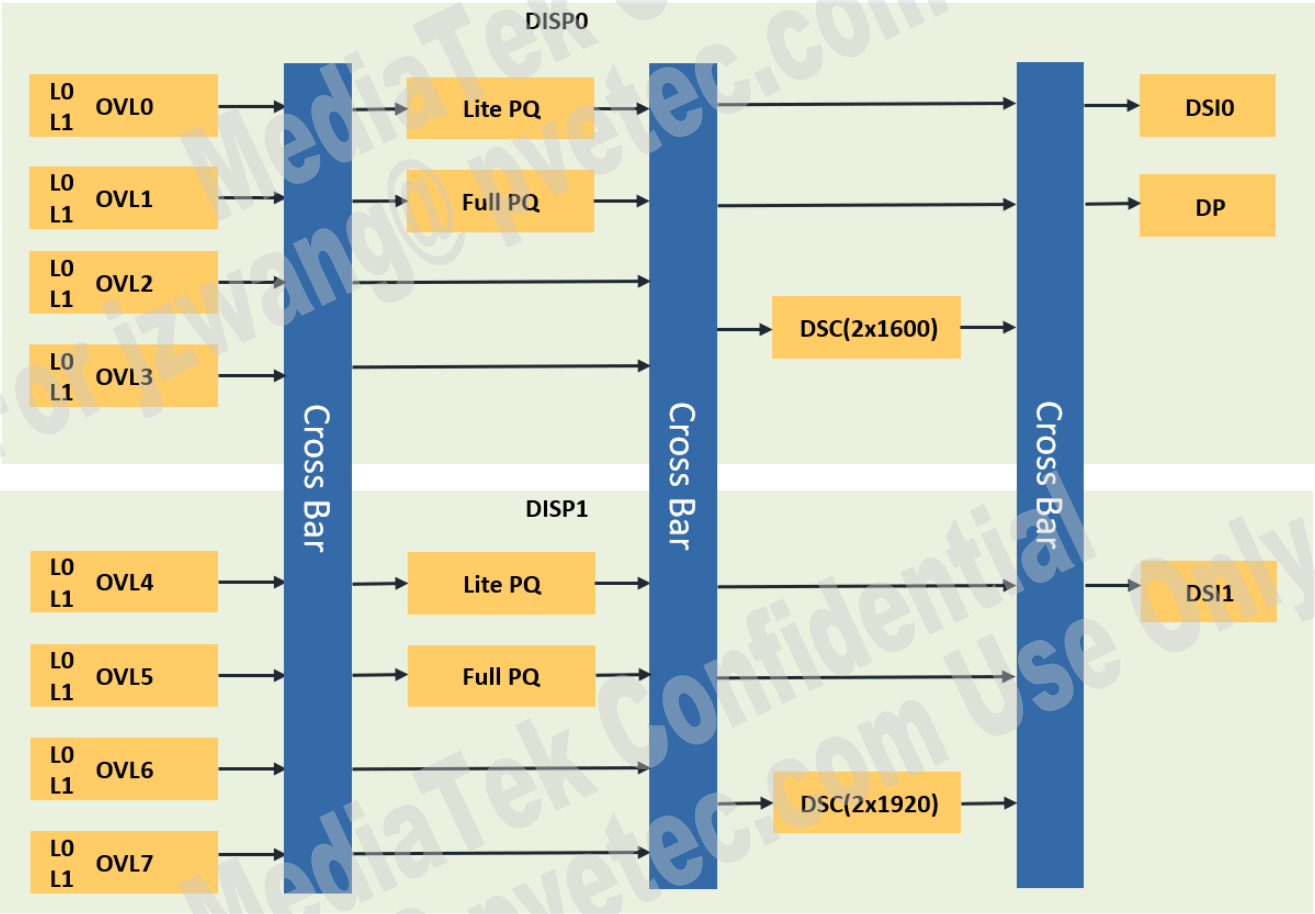


图 1-1. Display HW 架构图

1.2.3 SW 架构

1.2.3.1 Android

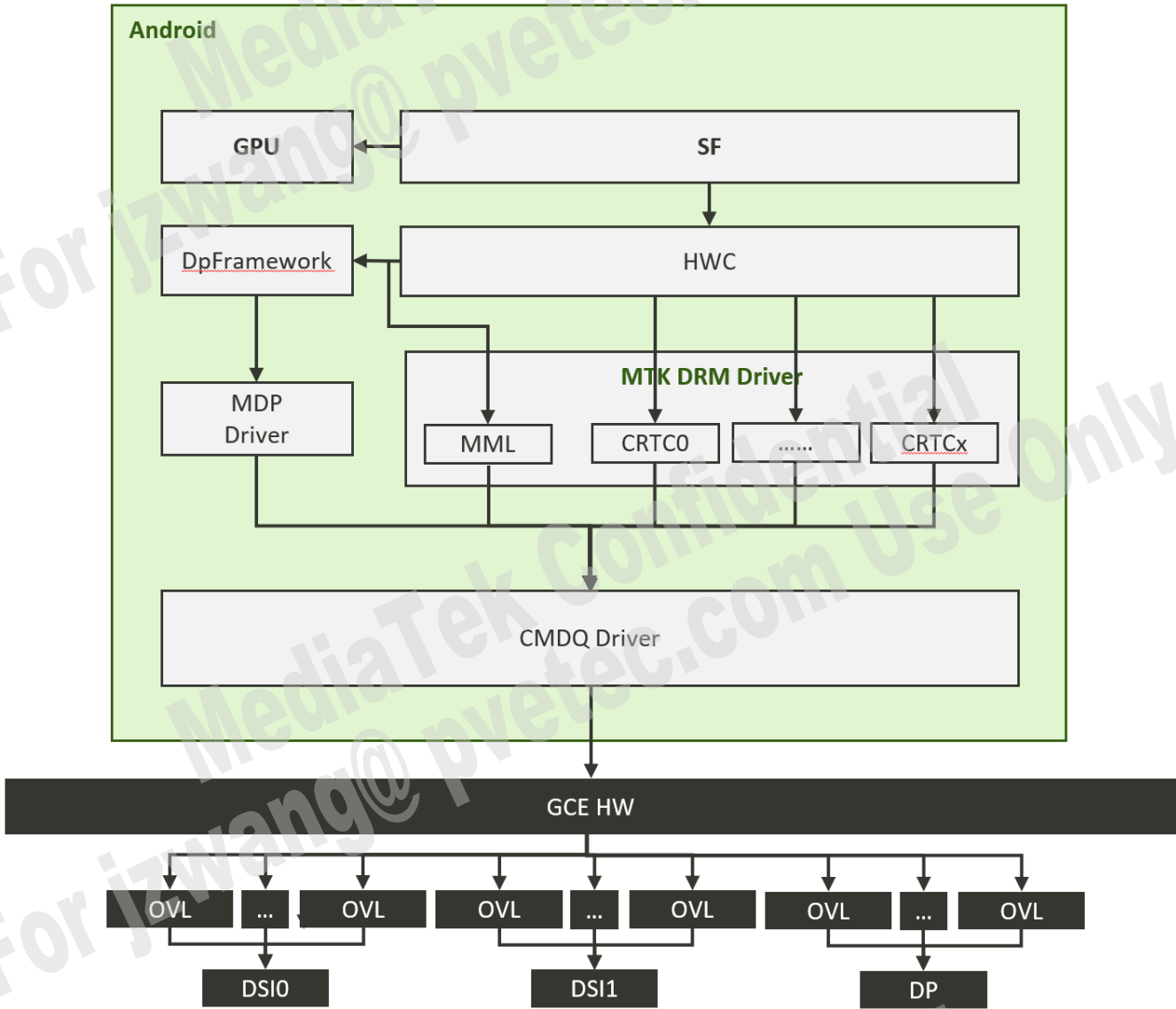


图 1-2. Display Android 架构图

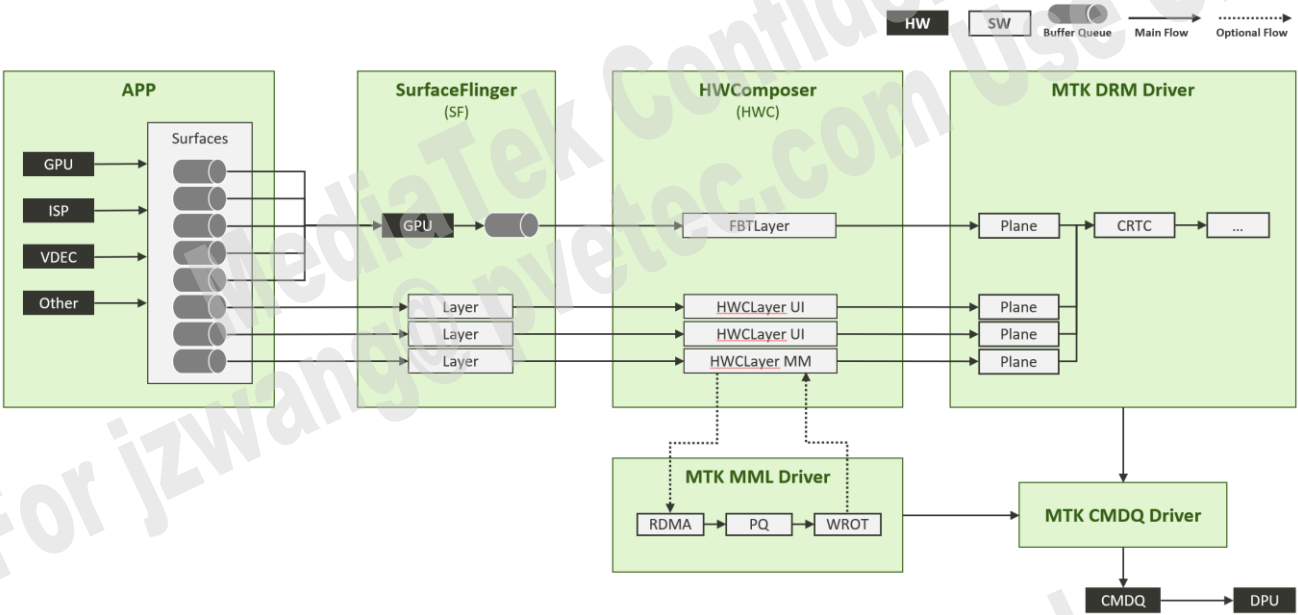


图 1-3. Android Display 流程

HWC 根据各个图层的属性、硬件能力和系统状态，决定每个图层由谁来合成。

设备合成（Device Composition/Hardware Composition）就是由 HWC 驱动的设备（即 OVL）直接合成。

GPU 合成（Client Composition/ GLES Composition）由 SurfaceFlinger 将图层交由 GPU 合成，合成结果再交给 HWC 作为一个图层。

简单说，HWC 会自下往上优先使用设备合成，超出 OVL 层数之后或遇到 OVL 不能处理的图层，则将该层及以上都使用 GPU 合成。

可以通过 `dumpsys SurfaceFlinger` 查看 HWC 的图层合成状况。如下图所示，`comp` 一列中显示 `DEV` 表示设备合成，`CLI` 表示 GPU 合成，`SOL` 即 `solid color`（纯色图层），是由 OVL 直接渲染并合成的，所以也是设备合成。

Display(0), active_config 0 brightness:0.030000 brightness_nits:26.400000 visible_layers:3 invisible_layers:1 commit_layers:2 ct_handle:0xb400007ae2201c0 alloc_id:2 unchanged:0 ct_fbdc:1 hbm:0 mml:1 disp_pq:0 mdp_disp_pq:0 mdp_disp_pq_query_support:0 partial_update_support:0

layer_id / zorder	handle / alloc_id	fmt	blend	comp	tr	ds	cl	pq	usage	gralloc status	caps	unchanged	inactive	mm scale
26 / 1	0xb400007ae2201c0 / 101	2	PRE	DEV	UI,DEV, HWCL: 468	0	0	142671872	110, 0,0,0,0,0, 0, 0	b00	0	40	0	010,0, 1.00, 1.00
11 / 2	0xb400007ae2201c0 / 26	1	PRE	CLI	INV,DEV, HWCL: 1684	0	0	142671872	110, 0,0,0,0,0, 0, 0	b00	0	0	0	010,0, 1.00, 1.00
10 / 3	0xb400007ae2201c0 / 31	1	PRE	CLI	INV,DEV, HWCL: 1684	0	0	142671872	110, 0,0,0,0,0, 0, 0	b00	0	0	0	010,0, 1.00, 1.00
layer_id / zorder	handle / alloc_id	fmt	blend	comp	tr	ds	cl	pq	usage	gralloc status	caps	mm scale		
27 / 0	0x0 / -1	0	NON	SOL	100,SOL, HWCL: 2517	0	0	142671872	1010, 0,0,0,0,0, 0, 0	0	0	0	0	010,0, 1.00, 1.00

1.2.3.2 Hypervisor (Yocto + Android + T-Box)

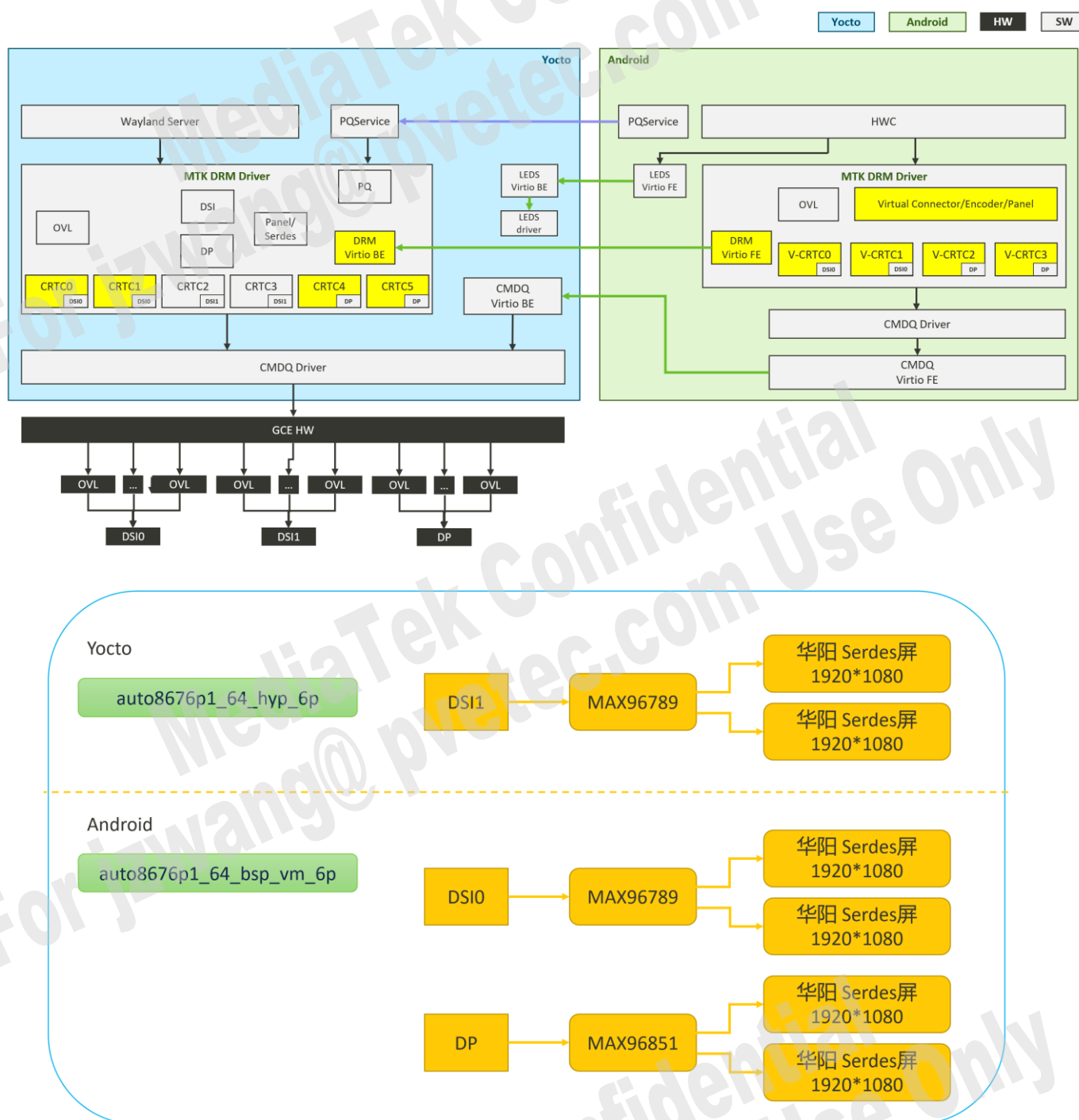


图 1-4. Display 多系统架构图

当前采用驱动层虚拟化方案，将 disp_pq 和 disp interface(DSI/DP) 虚拟化，OVL 仍是直通控制，兼顾性能。

完整的 CRTC 驱动（包括 ovl, disp_pq, dsi/dp）在 Host OS，Guest OS 采用 Virt-Connector/Encoder，通过 virtio 去 Host OS 的 DRM 驱动获取 display mode 以及开关 CRTC。Guest OS 更新图层则采用直通方式，直接驱动 OVL，无需和 Host 端通信。

以我司公版 project: auto8676p1_64_hyp_6p（Yocto 2 屏 + Android 4 屏）为例。在 Yocto Kernel 中，一共有 6 条 CRTC，CRTC0/1(DSI0) 和 CRTC4/5(DP)是 Android 显示，CRTC 2/3(DSI1)是 Yocto 显示。在 Android Kernel 中，connector/encoder 是虚拟的，在 dts 中的 node 是 virt_dsi0_0/virt_dsi0_1/virt_dp_0/virt_dp_1。

表 1-3. Connector 设备节点说明

设备节点	说明
dsi0 dsi1 dp_intf	host 端 connector 设备节点
virt_dsi0_0 virt_dsi1_0 virt_dp_0	guest 端 connector 设备节点
virt_dsi0_1 virt_dsi1_1 virt_dp_1	开启 superframe 后，第二屏的 connector 设备节点

1.3 配置/客制指南

1.3.1 设备节点配置

1.3.1.1 Host Yocto 端

请参考 MT8676_Android_Display_User_Manual_V1.0 在 Host 内核中配置所有的（Android + Yocto）多屏显示内核配置。

- Kernel config 路径：

meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/files/

- Kernel dts 路径：

meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/files/
src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/boot/dts/mediatek/

- Kernel ko table 路径：

meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/ko_order_table

可以参考我司公版 project: auto8676p1_64_hyp_6p：

- Kernel config 文件为

meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/files/auto8676p1_64_hyp_6p.defconfig

- Kernel dts 文件为

meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/files/auto8676p1_64_hyp_6p.dts
 meta/meta-mediatek-mt8676-hyp/recipes-kernel/linux/files/dtsi/cust_mt8676_display_config_hyp_6p.dtsi
 src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/boot/dts/mediatek/cust_mt8676_display_interface.dtsi

1.3.1.2 Guest Android 端

可以参考我司公版 project: auto8676p1_64_vm_6p:

Kernel dts 文件为:

src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/boot/dts/mediatek/cust_mt8676_display_config_vm_6p.dtsi

- 核心改动是 disable 掉原先代表实际硬件的 DSI/DP 相关节点, 并 enable virt_dsi/dp 节点。virt_xxx_1 表示该接口输出的第二个屏, 只有在开启 superframe 的情况下, 才需要 enable。

```
&dsi0 {
    status = "disabled";
};

&mipi_tx_config0 {
    status = "disabled";
};

&virt_dsi0_0 {
    status = "okay";
};

&virt_dsi0_1 {
    status = "okay";
    dsi-supply = <&virt_dsi0_0>;
};

&virt_dp_0 {
    status = "okay";
};

&virt_dp_1 {
    status = "okay";
    dp-supply = <&virt_dp_0>;
};
```

- 添加 CRTC 的 OVL 配置, 详见。

```
&dispsys config {
    crtc0-ovl-path = <DDP_COMPONENT_OVL0_2L>,
    <DDP_COMPONENT_OVL1_2L>,
    <DDP_COMPONENT_OVL2_2L>,
    <DDP_COMPONENT_OVL3_2L>,
    <DDP_COMPONENT_OVLSYS_DLO_ASYNC3>;
    crtc-layer-nr = <8>;
    pinctrl-names = "mode_te_te";
    pinctrl-0 = <&mtkfb_pins_lcm_dsi_te>;
    status = "okay";
};
```

1.3.2 dispsys_config 节点配置

DTS 中 dispsys_config 节点中需要定制化的部分主要是 OVL 的配置，即配置当前 OS 中每条 crtc 的硬件图层资源。

节点中的其他属性无需修改。

1.3.2.1 OVL 介绍

1. OVL 串接的顺序不能违反以下顺序：OVL4->OVL5->OVL6->OVL7->OVL0->OVL1->OVL2->OVL3，链路上靠前的 OVL z-order 更低。

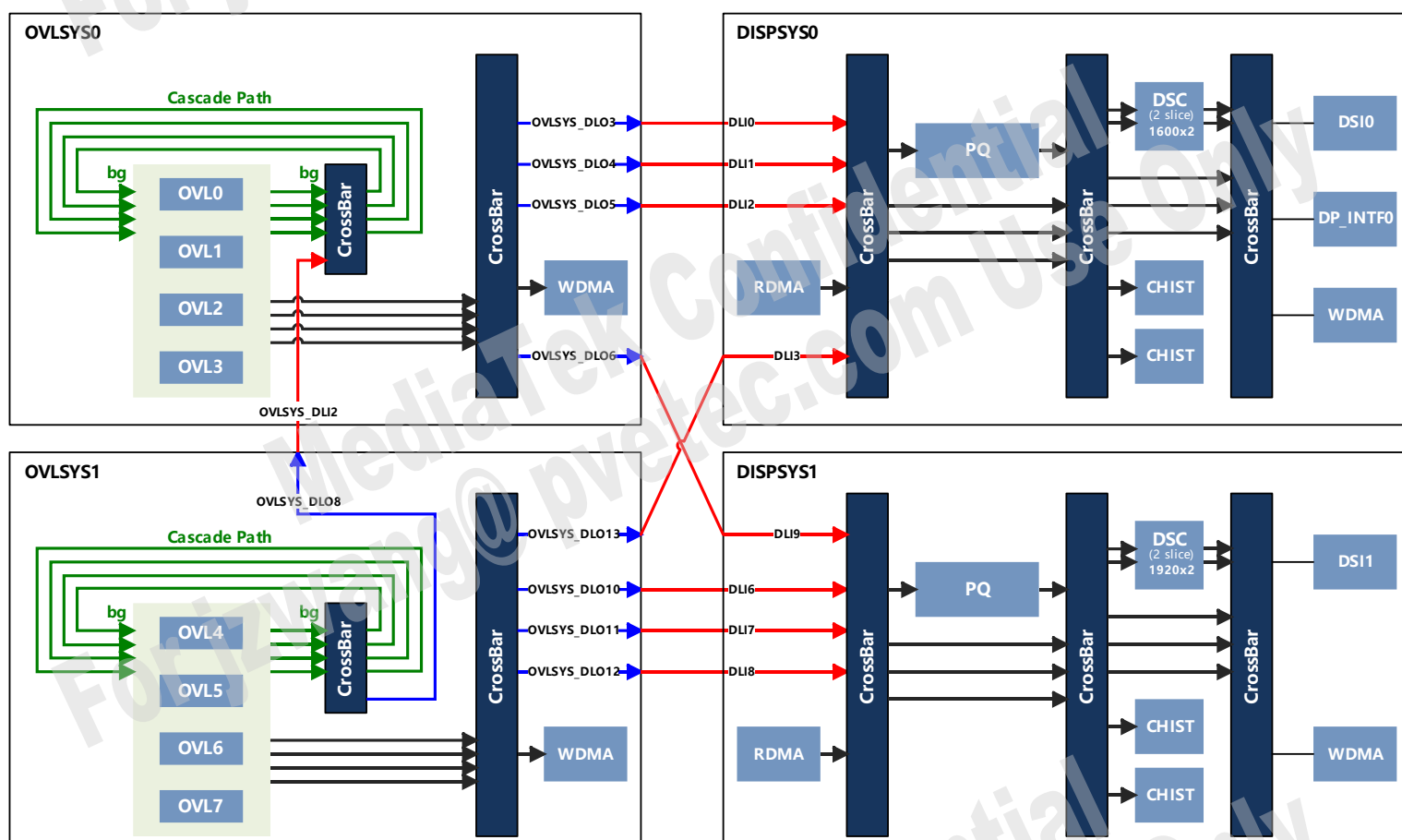


图 1-5. Display 硬件框图

显示链路跨越 OVSYS0/1 & DISPSYS0/1 需要通过 DLO/DLI。

OVSYS0/1 之间，只能 OVSYS1 通过 OVSYS_DLO8 和 OVSYS_DLI2 接到 OVSYS0。

OVSYS 和 DISPSYS 之间支持：

- 1) DSIO 显示链路走 OVSYS_DLO3 从 OVSYS0 到 DISPSYS0。

- 2) DSI1 显示链路走 OVLSYS_DLO10 从 OVLSYS1 到 DISPSYS1。
- 3) DP 显示链路走 OVLSYS_DLO6 从 OVLSYS0 到 DISPSYS1 或走 OVLSYS_DLO11 从 OVLSYS1 到 DISPSYS1（取决于最后一个 OVL 在 OVLSYS0 还是 OVLSYS1）。
2. 开 AFBC 的情况下，OVL 支持的最大宽度是 1920。
- 关 AFBC 的情况下，OVL 支持的最大宽度是 8191，但每个图层的最大内存带宽是 1GB/s。
- 超出限制，驱动默认会开启 multiple ovl pipes，使用多个 OVL 并行处理一个图层。
- 目前的软件中屏幕宽度超过 1920，drm 显示驱动中就会开启 multiple ovl pipes。
- 比如 2560x1600 的屏幕，用 2 ovl pipes；5120x1600 的屏幕，用 3 ovl pipes。

1.3.2.2 OVL 配置

OVL 通过如下三个属性配置：

1. crtc-ovl-path
- 表示当前 crtc 上的 OVL 链路。
- 链路上的 OVL 顺序不要违反以下前后顺序：OVL4->OVL5->OVL6->OVL7->OVL0->OVL1->OVL2->OVL3。
2. crtc-layer-nr
- 表示当前 crtc 上支持几个 plane。
- 假设 ovl pipes = n，则 1 个 plane 需要 n 个 OVL layer。
3. crtc-layer-table
- 表示当前 crtc 上的 plane 使用哪个 OVL Layer，即配置 plane 的 z-order。从 bit 0 ~ bit 15，每个 bit 代表一个 OVL layer，如下：

Bit	15	14	13	12	11	10	9	8
OVL layer	OVL7_L1	OVL7_L0	OVL6_L1	OVL6_L0	OVL5_L1	OVL5_L0	OVL4_L1	OVL4_L0
Bit	7	6	5	4	3	2	1	0
OVL layer	OVL3_L1	OVL3_L0	OVL2_L1	OVL2_L0	OVL1_L1	OVL1_L0	OVL0_L1	OVL0_L0

1.3.2.3 示例

这一节展示几个典型的多屏配置示例。

1.3.2.3.1 3 屏

多屏 & 图层需求如下：

屏幕规格	接口	图层需求
中控 2560x1440	DP	Linux Android Android Android Android Linux
仪表 1920x720	DSIO	Linux Android
HUD 800x480	DSI1	Linux Android

1. Yocto 端

```
&dispsys_config {
    // DP 作为 android 主屏。crtc0 是 DP, crtc1 是 DSI0, crtc2 是 DSI1。
    dp-is-main-display;

    // 表示 crtc0 的 OVL 链路是 OVL4->OVL5->OVL6->OVL1->OVL2->OVL3
    crtc0-ovl-path = <DDP_COMPONENT_OVL4_2L>,
                    <DDP_COMPONENT_OVL5_2L>,
                    <DDP_COMPONENT_OVL6_2L>,
                    <DDP_COMPONENT_OVLSYS_DLO_ASYNC8>,
                    <DDP_COMPONENT_OVLSYS_DLI_ASYNC2>,
                    <DDP_COMPONENT_OVL1_2L>,
                    <DDP_COMPONENT_OVL2_2L>,
                    <DDP_COMPONENT_OVL3_2L>,
                    <DDP_COMPONENT_OVLSYS_DLO_ASYNC6>;

    // 如果 DP 只用 OVL4->OVL5->OVL6, ovl-path 就会是如下:
    // crtc0-ovl-path = <DDP_COMPONENT_OVL4_2L>,
    //                  <DDP_COMPONENT_OVL5_2L>,
    //                  <DDP_COMPONENT_OVL6_2L>,
    //                  <DDP_COMPONENT_OVLSYS_DLO_ASYNC11>;

    /* 表示 yocto 端 crtc0 有 2 个 plane。
     * 因为 ovl pipes = 2, 所以 pipe0 使用 OVL6_L1 和 OVL4_L0 (最上层和最下层), 对应的 pipe1 使用
     * OVL3_L1 和 OVL1_L0。
     */
    // 同时 dp_intf 节点中需添加 host-os-use, weston 才能识别此 crtc。
    crtc0-layer-nr = <2>;
    crtc0-layer-table = <0x2100>;

    crtc1-ovl-path = <DDP_COMPONENT_OVL0_2L>,
                    <DDP_COMPONENT_OVLSYS_DLO_ASYNC3>;
    // 表示 yocto 端 crtc1 只有 1 个 plane, 使用 OVL0_L1 (最上层)。
    // 同时 dsi0 节点中需添加 host-os-use, weston 才能识别此 crtc。
    crtc1-layer-nr = <1>;
    crtc1-layer-table = <0x2>;

    crtc2-ovl-path = <DDP_COMPONENT_OVL7_2L>,
                    <DDP_COMPONENT_OVLSYS_DLO_ASYNC10>;
    // 表示 yocto 端 crtc2 只有 1 个 plane, 使用 OVL7_L1 (最上层)。
    // 同时 dsi1 节点中需添加 host-os-use, weston 才能识别此 crtc。
    crtc2-layer-nr = <1>;
    crtc2-layer-table = <0x8000>;
};
```

2. Android 端

```
&dispsys_config {  
    // DP 作为 android 主屏。crtc0 是 DP, crtc1 是 DSI0, crtc2 是 DSI1。  
    dp-is-main-display;  
  
    crtc0-ovl-path = <DDP_COMPONENT_OVL4_2L>,  
                    <DDP_COMPONENT_OVL5_2L>,  
                    <DDP_COMPONENT_OVL6_2L>,  
                    <DDP_COMPONENT_OVLSYS_DLO_ASYNC8>,  
                    <DDP_COMPONENT_OVLSYS_DLI_ASYNC2>,  
                    <DDP_COMPONENT_OVL1_2L>,  
                    <DDP_COMPONENT_OVL2_2L>,  
                    <DDP_COMPONENT_OVL3_2L>,  
                    <DDP_COMPONENT_OVLSYS_DLO_ASYNC6>;  
    /* 表示 android 端 crtc0 有 4 个 plane。  
     * 因为 ovl pipes = 2, 所以 pipe0 使用 OVL4_L1、OVL5_L0、OVL5_L1、OVL6_L0, 对应的 pipe1 使用  
     * OVL1_L1、OVL2_L0、OVL2_L1、OVL3_L0。  
     */  
    crtc0-layer-nr = <4>;  
    crtc0-layer-table = <0x1e00>;  
  
    crtc1-ovl-path = <DDP_COMPONENT_OVL0_2L>,  
                    <DDP_COMPONENT_OVLSYS_DLO_ASYNC3>;  
    // 表示 android 端 crtc1 只有 1 个 plane, 使用 OVL0_L0 (最下层)。  
    crtc1-layer-nr = <1>;  
    crtc1-layer-table = <0x1>;  
  
    crtc2-ovl-path = <DDP_COMPONENT_OVL7_2L>,  
                    <DDP_COMPONENT_OVLSYS_DLO_ASYNC10>;  
    // 表示 android 端 crtc2 只有 1 个 plane, 使用 OVL7_L0 (最下层)。  
    crtc2-layer-nr = <1>;  
    crtc2-layer-table = <0x4000>;  
};
```

1.3.2.3.2 4 屏 (DP Superframe)

多屏 & 图层需求如下:

屏幕规格	接口	图层需求
中控 2560x1440	DP superframe	Linux Android Android Android
副驾 2560x1440		Android Android
仪表 1920x720	DSI0	Linux Android
HUD 800x480	DSI1	Linux Android

1. Yocto 端

```
&dispsys_config {
    // DP 作为 android 主屏。crtc0&crtc1 是 DP, crtc2 是 DSI0, crtc3 是 DSI1。
    dp-is-main-display;

    /* 表示 crtc0 的 OVL 链路是 OVL4->OVL5->OVL6->OVL1->OVL2->OVL3
    * 此处是 superframe 的情况, 两个屏是按照 side by side 左右拼接后从 soc 输出。
    * 实际的 OVL 链路是 OVL4->OVL5->OVL6->OVL1->OVL2->OVL3。
    * crtc0 是左边屏, 用前四个 OVL4->OVL5->OVL6->OVL1, crtc1 是右边屏, 用后两个 OVL2->OVL3。
    */
    crtc0-ovl-path = <DDP_COMPONENT_OVL4_2L>,
                    <DDP_COMPONENT_OVL5_2L>,
                    <DDP_COMPONENT_OVL6_2L>,
                    <DDP_COMPONENT_OVLSYS_DLO_ASYNC8>,
                    <DDP_COMPONENT_OVLSYS_DLI_ASYNC2>,
                    <DDP_COMPONENT_OVL1_2L>,
                    <DDP_COMPONENT_OVL2_2L>,
                    <DDP_COMPONENT_OVL3_2L>,
                    <DDP_COMPONENT_OVLSYS_DLO_ASYNC6>;

    // 表示 yocto 端 crtc0 只有 1 个 plane。
    // 因为 ovl pipes = 2, 所以 pipe0 使用 OVL5_L1(最上层), 对应的 pipe1 使用 OVL2_L1。
    // 同时 dp_intf 节点中需添加 host-os-use, weston 才能识别此 crtc。
    crtc0-layer-nr = <1>;
    crtc0-layer-table = <0x800>;

    crtc1-ovl-path = <DDP_COMPONENT_OVL2_2L>,
                    <DDP_COMPONENT_OVL3_2L>;

    // 按需求, yocto 端是不需要 plane 的, 这里写 2 是兼容 drm, 写 0 会导致 drm 报错。
    // 注意 virt_dp_1 节点中不要添加 host-os-use, weston 就不会识别到此 CRTC。
    crtc1-layer-nr = <2>;

    crtc2-ovl-path = <DDP_COMPONENT_OVL0_2L>,
                    <DDP_COMPONENT_OVLSYS_DLO_ASYNC3>;
    // 表示 yocto 端 crtc2 只有 1 个 plane, 使用 OVL0_L1(最上层)。
    // 同时 dsi0 节点中需添加 host-os-use, weston 才能识别此 crtc。
    crtc2-layer-nr = <1>;
    crtc2-layer-table = <0x2>;

    crtc3-ovl-path = <DDP_COMPONENT_OVL7_2L>,
                    <DDP_COMPONENT_OVLSYS_DLO_ASYNC10>;
    // 表示 yocto 端 crtc3 只有 1 个 plane, 使用 OVL7_L1(最上层)。
    // 同时 dsi1 节点中需添加 host-os-use, weston 才能识别此 crtc。
    crtc3-layer-nr = <1>;
    crtc3-layer-table = <0x8000>;
};
```

2. Android 端

```
// DP 作为 android 主屏。crtc0 & crtc1 是 DP, crtc2 是 DSI0, crtc3 是 DSI1。
dp-is-main-display;

crtc0-ovl-path = <DDP_COMPONENT_OVL4_2L>,
                <DDP_COMPONENT_OVL5_2L>,
                <DDP_COMPONENT_OVL6_2L>,
                <DDP_COMPONENT_OVLSYS_DLO_ASYNC8>,
                <DDP_COMPONENT_OVLSYS_DLI_ASYNC2>,
                <DDP_COMPONENT_OVL1_2L>,
                <DDP_COMPONENT_OVL2_2L>,
                <DDP_COMPONENT_OVL3_2L>,
                <DDP_COMPONENT_OVLSYS_DLO_ASYNC6>;
/* 表示 android 端 crtc0 有 3 个 plane。
 * 因为 ovl pipes = 2, 所以 pipe0 使用 OVL4_L0、OVL4_L1、OVL5_L0, 对应的 pipe1 使用 OVL6_L0、
 * OVL6_L1、OVL1_0。
 */
crtc0-layer-nr = <3>;
crtc0-layer-table = <0x700>;

crtc1-ovl-path = <DDP_COMPONENT_OVL2_2L>,
                <DDP_COMPONENT_OVL3_2L>;
/* 表示 android 端 crtc1 有 2 个 plane。
 * 因为 crtc1 的全部 plane 都是给 android 用的, 所以不用显式写出 crtc-layer-table。
 * 因为 ovl pipes = 2, 所以 pipe0 使用 OVL2_L0、OVL2_L1, 对应的 pipe1 使用 OVL3_L0、OVL3_L1。
 */
crtc1-layer-nr = <2>;

crtc2-ovl-path = <DDP_COMPONENT_OVL0_2L>,
                <DDP_COMPONENT_OVLSYS_DLO_ASYNC3>;
// 表示 android 端 crtc2 只有 1 个 plane, 使用 OVL0_L0 (最下层)。
crtc2-layer-nr = <1>;
crtc2-layer-table = <0x1>;

crtc3-ovl-path = <DDP_COMPONENT_OVL7_2L>,
                <DDP_COMPONENT_OVLSYS_DLO_ASYNC10>;
// 表示 android 端 crtc3 只有 1 个 plane, 使用 OVL7_L0 (最下层)。
crtc3-layer-nr = <1>;
crtc3-layer-table = <0x4000>;

;
```

1.4 常见问题/故障排除

1.4.1 Dump 方法

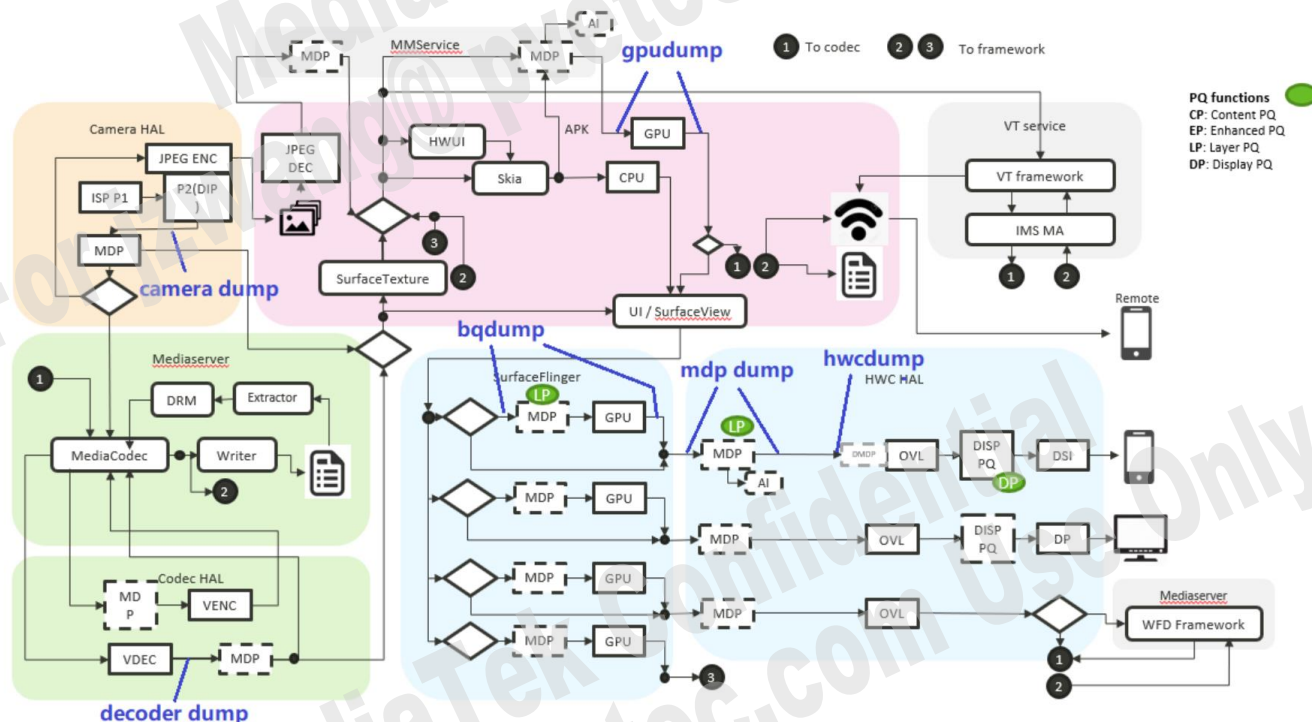


图 1-6. Dump 方法总览

当遇到花屏或者显示异常的问题时，可以通过显示链路上 dump 来确认是哪个环节产生的问题。

1.4.2 HWC CMD

- Log CMD:

```
adb shell setprop persist.vendor.debug.hwc.log V && adb shell setprop
vendor.debug.hwc.skip_log 0 && adb shell dumpsys SurfaceFlinger
```

- Dump sf info:

```
adb shell dumpsys SurfaceFlinger > sf.log
```

- Force GPU (GPU 叠图) :

```
adb shell service call SurfaceFlinger 1008 i32 1
```

1.4.3 显示驱动 CMD

On 都表示开, Off 表示关。

常用的 display mobile log:

```
adb shell "echo mobile:on > /d/mtkfb"
```

如果要抓开机 log, 需要直接改代码:

```
kernel/kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_debug.c
```

```
bool g_mobile_log = 1;
```

如果要开更详细的 log:

```
adb shell "echo detail:on > /d/mtkfb"
```

如果要抓开机 log, 需要直接改代码:

```
kernel/kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_debug.c
```

```
bool g_detail_log = 1;
```

Fence log:

```
adb shell "echo fence:on > /d/mtkfb"
```

如果要抓开机 log, 需要直接改代码:

```
kernel/kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_debug.c
```

```
bool g_fence_log = 1;
```

Irq log:

```
adb shell "echo irq:on > /d/mtkfb"
```

如果要抓开机 log, 需要直接改代码:

```
kernel/kernel_device_modules-6.1/drivers/gpu/drm/mediatek/mediatek_v2/mtk_debug.c
```

```
bool g_irq_log = 1;
```

抓 display diagnose dump:

```
adb shell "echo diagnose > /sys/kernel/debug/mtkfb && cat /sys/kernel/debug/mtkfb" > mtkfb.txt
```

1.4.4 Pattern

Pattern 命令需要在 Host OS 执行:

- dsi0 pattern:

```
adb shell "echo gce_wr:1400d178,c61,ffffffff > /sys/kernel/debug/mtkfb"
```

- dsi1 pattern:

```
adb shell "echo gce_wr:1420d178,c61,ffffffff > /sys/kernel/debug/mtkfb"
```

- dp pattern:

```
adb shell "echo gce_wr:1400bf00,41,ffffffff > /sys/kernel/debug/mtkfb"
```

1.4.5 黑屏

- 背光是否开启
- screencap 是否正确

```
android:  
adb shell screencap -d 0/1 /sdcard/1.png  
yocto:  
adb shell "source /data/wayland_env_file;XDG_PICTURES_DIR=/tmp/ weston-screenshooter"
```

- 如果有 bridge IC, bridge IC 是否正常
- DSI/DP pattern 是否显示正常

如果以上几点都 OK, 最后就需要 display owner 详细看 log 来定位问题。

1.4.6 花屏/闪屏

- 看 log 中是否有 DISP_OVL/RDMA underflow/abnormal

若有, 提供 display mobile log 和 display diagnose dump

- Force GPU 后是否正常

```
adb shell service call SurfaceFlinger 1008 i32 1
```

- DSI/DP pattern 是否正常

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。